

# Modern adatbázis-rendszerek (MSc)

## 7. Gyakorlat: LINQ programozás

GitHub: [https://github.com/KissKonradUni/CNY8MP\\_MDBPract](https://github.com/KissKonradUni/CNY8MP_MDBPract)

# A LINQ (Language Integrated Query)

Az adatlekérdezés nyelvi szintre emelése: hagyományosan az algoritmus és az adatlekérdezés (SQL) két különálló dolog, a LINQ viszont hidat képez a kettő között.

- **Egységes megközelítés:** Ugyanaz a lekérdező szintaxis használható memóriabeli objektumhalmazokon, relációs adatbázisokon vagy épp hierarchikus XML-fákon.
- **Deklaratív stílus:** Ahelyett, hogy meghatároznánk, *hogyan* lépkedjen végig a ciklus az elemeken, csupán feltételeket fogalmazunk meg. (SQL-analógia).

# Fontos C# koncepciók

Mielőtt XML-fájlokat elemeznénk, három modern C# nyelvi elemet kell ismernünk:

1. **Típus-következtetés (var):** A C# erősen típusos nyelv, de a fordító képes kikövetkeztetni a pontos típust az egyenlőségjel jobb oldalából, így a kód olvashatóbbá válik.
2. **Lambda-kifejezések (névtelen függvények):** A Where és Select metódusok paraméterként úgynevezett delegáltakat (függvényreferenciákat) várnak.
  - Szintaxis: (változó) => logikai\_kifejezés
3. **Fluent API:** A funkcionális programozásból ismert láncolási technika.
  - Példa: `készlet.Select(...).Where(x => x.Készlet > 0).ToList()`

# 0. Feladat: A fejlesztőkörnyezet előkészítése

A .NET parancssoros eszközkészlete (CLI) felel a projektstruktúra létrehozásáért. A projekthez használjunk egy almappát (pl. NEPTUNKOD\_0325/etterem).

```
1 # Új konzolos .NET projekt inicializálása
2 dotnet new console
3 # A projekt fordítása és futtatása
4 dotnet run
```

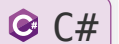


**Fontos:** A feldolgozandó XML fájlt a megnyitott projekt munkakönyvtárába (a projekt gyökérkönyvtárába) kell helyezni.

# 1. Feladat: Az XML-dokumentum beolvasása memóriába

A `System.Xml.Linq` névtér modern DOM-kezelést biztosít. A klasszikus W3C DOM modellel szemben (amelyet pl. Java-ban ismerhettetek meg), sokkal kevesebb “boilerplate” kódot igényel.

```
1 using System.Xml.Linq;
2 using System.Linq;
3
4 // A teljes dokumentum betöltése a fájlrendszerből
5 XDocument dokumentum = XDocument.Load("etterem.xml");
6 // "Leszármazott" elemeket a .Descendants() hívással kérdezzük le.
7 XElement gyoker = dokumentum.Descendants("vendeglatas").First();
```



A `.First()` metódus azt biztosítja, hogy a lekérdezett gyűjteményből az attribútumokkal rendelkező legelső tényleges XML node-ot kapjuk meg objektumként.

Ez így önmagában természetesen nem teljesen biztonságos (nem kezeljük a hiányzó vagy hibás értékű elemeket), de a gyakorlat kedvéért most elfogadható.

## 2. Feladat: Deklaratív szűrés

Az első igazi LINQ-lekérdezés bevezetése. Célunk az elemek szűrése attribútumok vagy gyermekelemek értéke alapján.

```
1 Console.WriteLine("(1.) Az ötcsillagos éttermek kilistázása:\n");
2
3 var otCsillagosEttermek = gyoker.Descendants("etterem")
4     .Where(elem => elem.Descendants("csillag").First().Value == "5")
5     .ToList();
6
7 otCsillagosEttermek.ForEach(elem =>
8     Console.WriteLine(" - " + elem.Descendants("nev").First().Value)
9 );
```



Megjegyzés: a `.value` tulajdonság beolvasása minden esetben `string` típust eredményez, ezért az összehasonlítás (`==`) is sztringként (`"5"`) történik.



### 3. Feladat: Komplex relációs összekapcsolás

**Probléma:** Egy szabványos egyed-kapcsolat rendszerben az azonosítók (kulcsok) csak hivatkoznak egy másik entitásra.

SQL-ben ez egy klasszikus JOIN feladat. LINQ és XML esetén ezt beágyazott kereséssel és névtelen (anonymous) típusok visszaadásával oldjuk meg:

1. Végigmegyünk a rendezes elemeken.
2. A rendeléshez tartozó idegen kulcsokat kinyerjük (`elem.Attribute(...)`).
3. Ezekkel a kulcsokkal kikeresünk a teljes dokumentumból az éttermet és a vendéget.
4. Projektáljuk az eredményt (`Select -> new { ... }`).

### 3. Feladat: Az összeállított lekérdezés

```
1  var joinEredmeny = gyoker.Descendants("rendeles")
2      .Select(elem => {
3          var vendegID = elem.Attribute("e_v_v").Value;
4          var vendegElem = gyoker.Descendants("vendeg")
5              .First(v => v.Attribute("vkod").Value == vendegID);
6          var vendegNev = vendegElem.Descendants("nev").First().Value;
7          var osszeg = elem.Descendants("osszeg").First().Value;
8          return new { Vendeg = vendegNev, Osszeg = osszeg };
9      })
10     .ToList();
```



## 4. Feladat: Aggregáció és konverziók

Statisztikai vagy üzleti összesítések (pl. átlagos fogyasztás, teljes bevétel) számítása beépített LINQ-metódusokkal lehetséges (Average, Sum, Max, Min).

Az XML-dokumentumokban az értékeket szöveges (string) formában tároljuk, ezért az aggregáció előtt gondoskodni kell a konverzióról.

```
1 var atlagKoltes = gyoker.Descendants("rendeles")
2     // Közvetlenül string kollekcióra projektálunk
3     .Select(rendeles => rendeles.Descendants("összeg").First().Value)
4     // Átalakítjuk double-lé és átlagoljuk
5     .Average(osszeg => double.Parse(osszeg));
6
7 Console.WriteLine($"Az átlagos költség: {atlagKoltes} Ft");
```



## 5. Feladat: Módosítás

Bár a LINQ alapvetően mellékhatásmentes (“pure function”) jellegű, a hierarchikus DOM-objektumfáknál helyben (in-place) módosítást is alkalmazhatunk.

```
1  gyoker.Descendants("etterem")
2      .Where(elem => elem.Descendants("csillag").First().Value == "3")
3      .ToList()
4      .ForEach(elem => {
5          elem.Remove(); // Teljes node törlése az XML-fából
6      });
7
8  // A memóriában megváltozott struktúra mentése
9  XDocument toroltDokumentum = new XDocument(gyoker);
10 toroltDokumentum.Save("etterem_torolt.xml");
```



# Gyakorlati önálló (házi) feladat

**A hallgatói feladat:** Az előbb bemutatott, 5 lépésből álló életciklust (Beolvasás → Szűrés → Join → Aggregáció → Adatmanipuláció és mentés) implementálni a **saját témájú**, az előző hetekben használt XML-dokumentumon.

A beadással és a repo-struktúrával kapcsolatos elvárások:

- Két külön, dedikált mappa legyen létrehozva a NEPTUNKOD\_0325 könyvtár alatt a teljes izoláció érdekében (egy az órai, egy a házi programhoz).
- A fordító bin/ és obj/ mappái szigorúan **gitignore** szabály alá esnek; GitHubra csak a forráskódot töltsük fel.